내 칩 과제 5

한찬희

과제8: 버튼 입력의 확인이 화면의 갱신에 맞춰져야 하는 이유를 찾아 그 대책을 세우라

-버튼 입력이 화면의 갱신 주기(Refresh Rate)와 동기화되지 않을 경우, 입력 신호의 샘플링 속도와 화면 출력 속도 간의 불일치로 인해 '입력 누락'이나 '과도한 이동' 현상이 발생한다. 현재 시스템에서 화살표 키를 누를 때 포대가 급격히 이동하는 이유는, 아주 빠른 클럭 주파수(예: 50MHz)에서 입력 신호를 그대로 처리하여 좌표 계산에 반영하기 때문이다. 화면은 초당 60회(60Hz)만 갱신되는데, 그 사이의 수많은 클럭 동안 입력값이 계속 누적되거나 반영되면 사용자 의도보다 훨씬 빠르게 오브젝트가 이동하게 된다.

이를 해결하기 위해 먼저 버튼 입력을 시스템 클럭에 동기화하고, 디바운싱을 통해 버튼의 채터링 현상을 제거해야 한다. 이후 화면의 프레임 갱신 시점에 맞춰 이동 제어 신호를 생성하여 한 프레임당 포대의 좌표를 일정한 값만큼 변경하도록 제한한다. 또한 짧은 버튼 입력이 누락되지 않도록 입력 이벤트를 별도로 저장하는 방식도 적용할 수 있다.

이와 같이 버튼 입력의 감지와 오브젝트의 좌표 갱신을 분리하면 입력 누락과 과도한 이동을 방지하고, 화면 갱신 주기에 맞춰 안정적인 움직임을 구현할 수 있다.

과제9: 비디오 신호에서 V_Sync 필요성에 대하여 논하라.

비디오 신호에서 V_Sync는 한 프레임의 끝과 다음 프레임의 시작을 알리는 수직 동기 신호이다. 디스플레이는 H_Sync로 각 라인의 시작을 맞추지만, 라인이 몇 번째이며 어디서 프레임이 새로 시작되는지는 V_Sync로만 알 수 있다. V_Sync가 없으면 수신 측은 프레임 경계를 찾지 못해 영상이 수직 방향으로 흐르거나 롤링되어 안정적으로 표시되지 않는다. 또한 V_Sync가 포함된 수직 블랭킹 구간은 CRT의 빔 복귀와 프레임 버퍼 갱신에 필요한 시간을 제공하고, 주사율을 결정한다. V_Sync는 영상의 수직적 안정성과 프레임 단위 동기를 보장하는 신호이다.

과제10: 시뮬레이션에 그치지 않고 FPGA 에뮬레이션 검증을 해야하는 이유를 설명해보라.

시뮬레이션은 논리적 정확성을 검증하는 데 유리하지만, 소프트웨어가 하드웨어 동작을 모사하므로 속도가 매우 느려 대규모 시스템이나 실제 동작 환경을 검증하기에는 한계가 있다. FPGA 에뮬레이션 검증이 필요한 이유는 다음과 같다.

첫째, 실시간에 가까운 속도로 장시간 동작을 검증할 수 있다. 시뮬레이션이 수 Hz~수 kHz 수준이라면 FPGA 에뮬레이션은 수 MHz 이상으로 동작하므로, OS 부팅이나 드라이버 구동 같은 소프트웨어 통합 테스트와 오래 걸리는 시나리오를 현실적인 시간 안에 수행할 수 있다.

둘째, 실제 I/O와 멀티 클럭 도메인 환경을 검증할 수 있다. 디스플레이, 버튼 같은 실물 주변장치와 연결하고, 서로 다른 클럭 사이의 CDC 문제를 실제 물리 타이밍으로 확인할 수 있다.

셋째, 시뮬레이션이 놓친 버그를 발견할 수 있다. 시뮬레이션에서는 도달하기 어려운 드문 상태, 타이밍 경쟁, 테스트벤치가 가정하지 못한 입력이 실제 동작에서 드러난다. 다만 FPGA는 ASIC과 타이밍 특성이 다르고 내부 신호 관찰이 어렵다는 한계가 있으므로, 에뮬레이션은 시뮬레이션을 대체하기보다 보완하는 단계로 병행해야 한다.

과제11: {인공지능}은 DUT의 내부 신호를 추적하기 어렵다는 점을 하드웨어 에뮬레이션의 치명적 단점으로 꼽는다. 상용의 하드웨어 가속기(Synopsys ZeBu, Cadence Palladium 등)은 어떤 대책을 두고 있는지 찾아보라.

하드웨어 에뮬레이터는 수백만 개의 게이트를 병렬로 처리하기 위해 내부 로직을 고도의 압축된 구조로 구현하므로, 시뮬레이터처럼 모든 내부 노드의 신호를 실시간으로 추출하는 것이 매우 어렵다.그러나 상용 가속기는 이 약점을 여러 방식으로 완화하고 있다.

먼저 상용 장비는 컴파일 단계에서 관찰할 신호용 프로브 로직을 하드웨어에 삽입하고, 실행 중 값을 온칩 메모리에 기록한 뒤 업로드하는 방식으로 신호 가시성을 확보한다. Cadence Palladium의 FullVision은 모든 넷을 실행 속도 그대로 볼 수 있게 하고, 동적 프로브로 선택한 신호를 대량의 샘플로 내려받게 한다. Synopsys ZeBu Server-3도 전체 신호 가시성을 제공한다. 수집된 신호는 Verdi 같은 시뮬레이션용 디버그 환경으로 불러와 파형과 트랜잭션 수준에서 분석하며, 프로토타이핑 계열인 Synopsys HAPS의 Deep Trace Debug는 수천 개 신호를 속도 저하 없이 깊은 버퍼에 담고 런타임에 트리거를 설정해 VCD나 FSDB로 내보낼 수 있다.

여기에 더해 특정 시점의 상태를 저장했다가 복원하는 체크포인트 기능과 같은 입력으로 다시 실행하는 결정적 재실행이 쓰인다. 이를 통해 오류가 발생한 구간만 골라 상세 신호를 얻을 수 있으며, ZeBu의 deterministic rerun, Palladium의 체크포인트 복원, Siemens Veloce의 재생·복원이 이에 해당한다.

과제12: 에뮬레이션 검증 킷의 하드웨어는 PSCE-TRANS 보드와 PSCE-MI 보드를 결합한 구성이다. 각 보드의 회로느는 아래와 같다. (a) 쿼터스 합성 스크립트[링크]에서 PSCE-TRANS 보드에 장착된 FPGA의 부품명을 확인하고 입출력 핀 할당과 비교하라. (b) 프로토 타이핑 펌웨어[링크]와 에뮬레이션 모델링 인터페이스 펌웨어[링크]를 비교하라.

(a) FPGA 부품명 확인 및 입출력 핀 할당 비교

제공된 쿼터스(Quartus) 합성 스크립트(space_invaders_engine_glcd.tcl)를 분석한 결과, PSCE-TRANS 보드에 장착된 FPGA의 부품명은 EP4CE6E22C8 (Cyclone IV E 제품군)임을 확인할 수 있다.

스크립트 내의 set_location_assignment 명령어를 통해 정의된 입출력 핀 할당은 다음과 같다
Clock 및 제어 신호: clk(PIN_135), clk_o(PIN_119), rst_n(PIN_114)
버튼 입력: btn_left(PIN_112), btn_right(PIN_24), btn_fire(PIN_111)
LCD 제어 및 데이터: lcd_write_en(PIN_120), lcd_data(PIN_121)
LCD 좌표 데이터: lcd_x[6:0] (PIN_132, 133, 136, 137, 138, 141, 142), lcd_y[5:0] (PIN_124, 125, 126, 127, 128, 129)
하드웨어 설계 시 물리적인 FPGA 핀과 보드상의 회로를 직접 연결하기 위한 매핑 정보를 포함하고 있다.

(b) 프로토 타이핑 펌웨어[링크]와 에뮬레이션 모델링 인터페이스 펌웨어[링크]를 비교하라.

두 펌웨어는 같은 FPGA(DUT)를 상대로 하며, 게임 로직은 FPGA 안의 DUT가 실행한다. 두 펌웨어는 모두 RP2040 MCU에서 DUT의 입출력을 처리해 OLED에 표시하는 주변장치 역할을 한다.

공통적으로 128×64 프레임버퍼(TableBMP)에 주소(y×16 + x/8)와 비트(0x80>>(x%8))를 계산해 픽셀을 저장하고, (127,63)에 도달하면 갱신 플래그를 세워 core1이 화면을 그린다. 차이는 DUT와 접속하는 방식에 있다.

동작 환경의 차이

프로토타이핑 펌웨어: MCU가 FPGA와 물리 핀으로 직접 연결된다. 각 신호가 PIN_CLK, PIN_LCD_X_0 같은 개별 GPIO로 정의되어 있고, RP2040_PWM으로 100kHz 클럭을 PIN_CLK에 공급한다. U8g2는 firstPage/nextPage 페이지 방식으로 OLED에 그린다.

에뮬레이션 모델링 인터페이스 펌웨어: 이 펌웨어도 MCU에서 실행되며, PSCE_Config.h의 PSCE 객체를 통해 초기화, 패킷 송수신, 디스플레이(psce.u8g2)를 처리한다. 개별 핀 번호 대신 입출력 비트맵 규약으로 DUT에 접근하고, 클럭 방식은 DUT_CLK_BITMAP(0x00은 THREAD, 0x10은 METHOD 에뮬레이션)으로 선택한다. 화면은 clearBuffer/sendBuffer의 풀버퍼 방식이다.

데이터 통신 방식의 차이

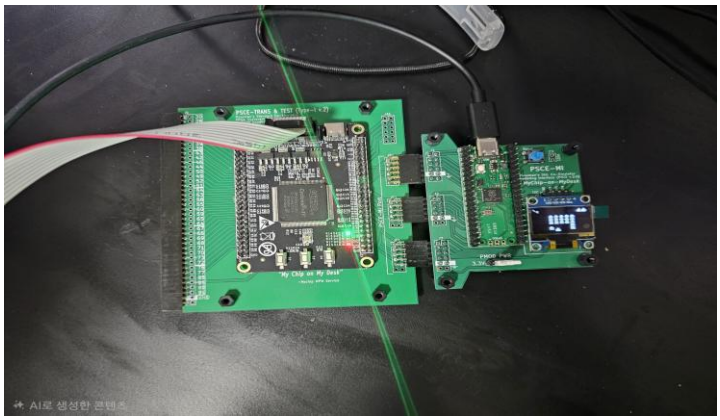
프로토타이핑 펌웨어: PC에서 시리얼로 받은 바이트를 digitalWrite로 rst_n과 버튼 핀에 출력해 DUT 입력을 만든다. DUT 출력(lcd_x, lcd_y, lcd_data)은 digitalRead로 읽고, clk_o 상승 에지 인터럽트(handlerLCD_DATA)가 발생할 때 픽셀을 수집한다.

에뮬레이션 모델링 인터페이스 펌웨어: RxPacket으로 입력 1바이트(N_RX=1, clk, rst_n, 버튼)를, TxPacket으로 출력 2바이트(N_TX=2, byte0: lcd_write_en, lcd_data, lcd_y[5:0] / byte1: lcd_x[6:0])를 주고받는다. 픽셀 갱신 시점은 PIN_IO_REQ 상승 에지 인터럽트(handlerIO_Req)가 정한다. 신호가 바이트 패킷으로 묶여 전달된다는 점이 핵심이다.

사용 목적의 차이

프로토타이핑 펌웨어는 FPGA에 올린 설계를 핀 수준의 신호로 직접 구동하여 실물 프로토타입으로 동작을 확인하는 데 목적이 있다.

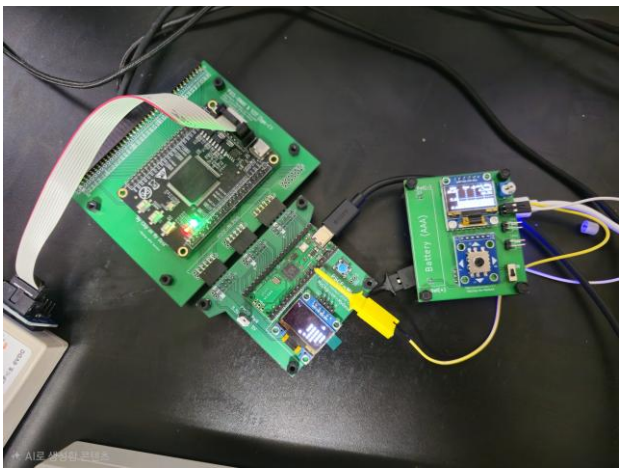
에뮬레이션 모델링 인터페이스 펌웨어는 같은 FPGA를 PSCE 패킷 인터페이스로 제어하며, 입출력과 클럭 방식을 비트맵 규약으로 정의해 DUT를 검증하는 데 목적이 있다.



처음시뮬레이션의 결과이다. 컴퓨터와 장치의 게임이 동시에 나왔다.

라즈베리 파이에 upload_mi에 연결하는데 제일 많은 시간을 썼다. Bootsel을 이해하는데 시간이 걸리기도 했고 학습자료를 참고해 mount e : /mnt/e를 계속 해봤으나 안됐다 알고보니 /mnt/pico_boot로 되어 있었다.

컴퓨터 화면과 장치에 모두 동작하려니 굉장히 느리게 작동하였다.



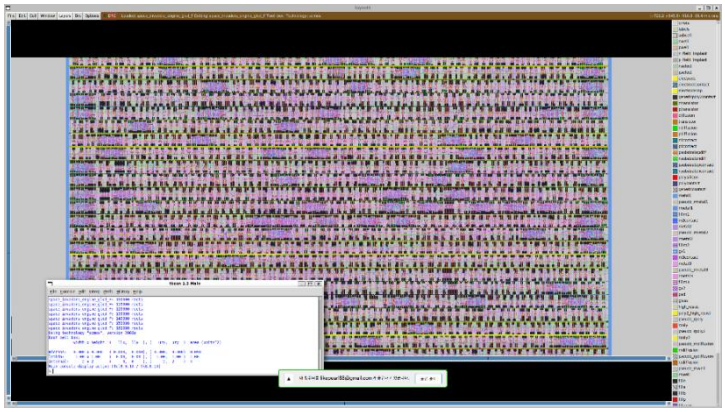
장치에만 에뮬하니 정상적으로 작동하였다.

Q.14.애서 똑같이 emulation을 해보는데 upload가 되지 않았다.

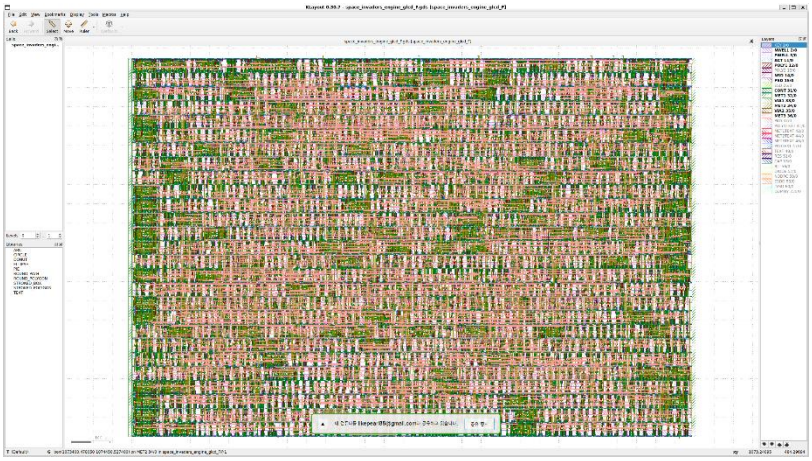
Q.14 simulation에서 make build에 많은 오류가 있었다. vpi_stub.cpp파일에 sc_space_invaders_engine_glcd로 되어있어야할 이름들이 sc_brick_out으로 되어있었다.

파일들이 다른게임을 기준으로 적힌 것이 많았다. 또한 sc_space_invaders_engine_glcd_TB.h 이파일 안에서도 선언이 잘못된 것이 많았다. 너무 많아 ai의 도움을 받아 모두 수정하였더니 build와 run 모두 작동하였다.

다음은 ETRI050에 있는 make 파일을 전부 실행해주고 layout에 있는 make 파일들도 모두 실행해주었다.그리고



magic



klayout

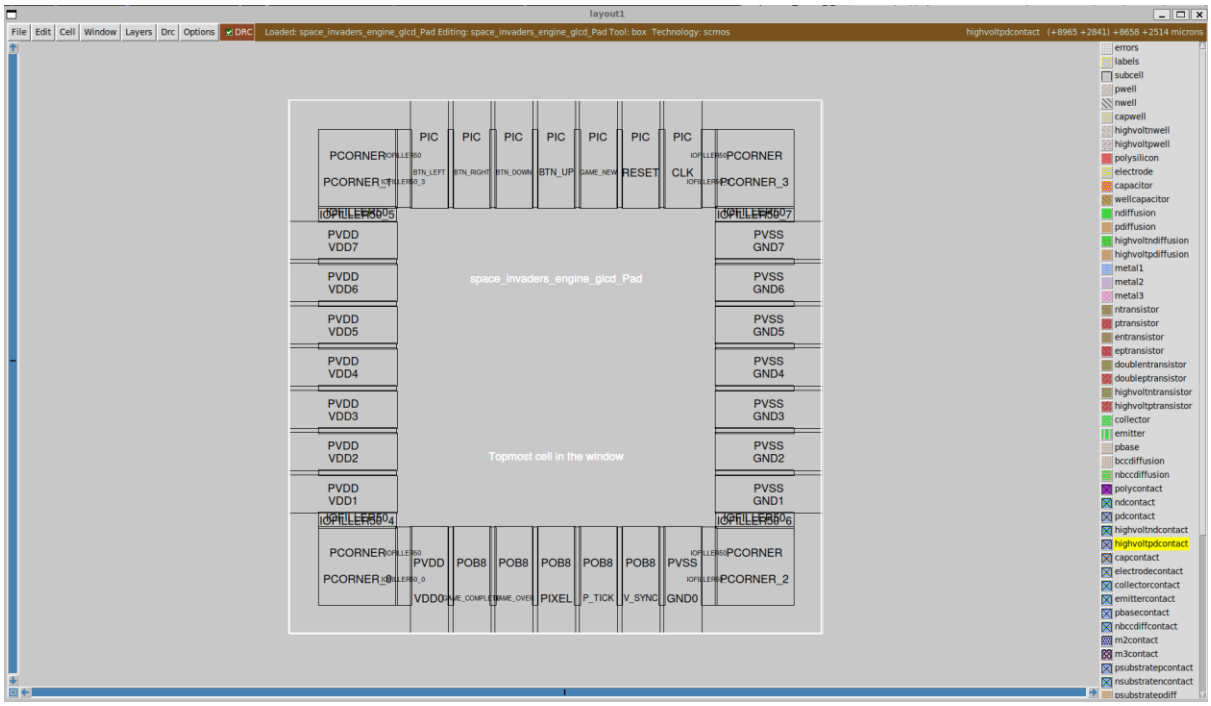


Magic ~_Top.mag 의 결과

Root cell box:

width x height (llx, lly), (urx, ury) area (units^2)

microns: 4872.000 x 3283.500 (-31.000, -12.000), (4841.000, 3271.500) 15997212.000
lambda: 4872.00 x 3283.50 (-31.00, -12.00), (4841.00, 3271.50) 15997212.00
internal: 9744 x 6567 (-62, -24), (9682, 6543) 63988848



Magic ~pad.mag의 결과

% select top cell Topmost cell in the window
% box
Root cell box:

width x height (llx, lly), (urx, ury) area (units^2)
microns: 5683.000 x 5683.000 (307.000, 327.000), (5990.000, 6010.000) 32296488.000
lambda: 5683 x 5683 (307, 327), (5990, 6010) 32296489
magic top과 pad는 같은 결과를 볼 수 있었다.

PinMeg는 볼 수 없었다.

make generate_gds

~/ETRI050_DesignKit/scripts/generate_chip.sh space_invaders_engine_glcd ₩

97.5 97.5 ₩

usage: generate_chip <core name> <pad_x> <pad_y> <route_x> <route_y> <core_x> <core_y>

make: *** [Makefile:75: space_invaders_engine_glcd_Top.gds] Error 2
pad의 결과 만 확인할 수 있었고 pin과 core, route의 결과는 빠져있었다. 그래서 생성할 수 없었다.